

目录

- AI 透析中心管理系统 (AI-HMS) —项目交接文档 2
 - 目录 2
- 快速开始 2
- 第 1 章 项目概述 3
- 第 2 章 技术架构 3
- 第 3 章 环境搭建 4
 - 3.1 前置要求 4
 - 3.2 前端 4
 - 3.3 后端 5
 - 3.4 测试登录 5
- 第 4 章 项目结构 5
 - 4.1 前端 5
 - 4.2 后端 6
- 第 5 章 双 API 架构说明 6
 - 5.1 REST API (Go 新后端—主力) 6
 - 5.2 GraphQL API (HDIS 旧后端—待迁移) 7
 - 5.3 Mock 数据 (Fallback) 7
- 第 6 章 数据库说明 7
 - 6.1 基本信息 7
 - 6.2 核心模型文件 7
 - 6.3 ID 策略 8
- 第 7 章 功能完成度与后续工作量 8
 - 7.1 整体进度 8
 - 7.2 按页面完成度 9
 - 7.3 下一步该做什么 (按优先级) 9
 - 7.4 后续工作量估算 10
- 第 8 章 已知问题与注意事项 10
 - 8.1 测试服务器 vs 医院线上系统存在差异 10
 - 8.2 HDIS 集成特殊性 10
 - 8.3 登录认证为硬编码 10
 - 8.4 数据库注意事项 11
 - 8.5 前端 UI 变动遗留 11
 - 8.6 高风险问题清单 (P0) 11
- 第 9 章 后续开发建议 11
 - 9.1 推荐的开发顺序 11
 - 9.2 技术要点 11
 - 9.3 AI 辅助开发 11
 - 9.4 角色权限参考 12
- 附录 A 产品功能地图 12
 - A.1 路由级总览 12
 - A.2 按页面详细功能 13
- 附录 B 后续开发计划 17
 - B.1 阶段划分 17
 - B.2 后端模块缺口 18

B.3 全量任务表 (68 条) 18
B.4 工时统计 20
B.5 依赖关系 21
B.6 风险管理 22

AI 透析中心管理系统 (AI-HMS) —项目交接文档

交接日期: 2026-03-09 项目状态: 开发中 (整体完成度约 39%) 文档版本: 1.0

目录

- 快速开始
- 第 1 章 项目概述
- 第 2 章 技术架构
- 第 3 章 环境搭建
- 第 4 章 项目结构
- 第 5 章 双 API 架构说明
- 第 6 章 数据库说明
- 第 7 章 功能完成度与后续工作量
- 第 8 章 已知问题与注意事项
- 第 9 章 后续开发建议
- 附录 A 产品功能地图
- 附录 B 后续开发计划

快速开始

接手后 5 分钟内搞清楚项目现状和下一步。

第一步：跑起来看一眼

```
cd ai-hms-frontend && npm install && npm run dev
```

打开 http://localhost:5173，不需要后端，系统会自动使用 Mock 数据。用这一步先熟悉 UI 和业务流程。

第二步：理解现状

指标	数值
总功能数	46 个
已完成（前后端对接）	18 个（39%）
仅前端 UI（假数据）	24 个（52%）
部分完成	4 个（8%）
后端 API 已就绪但前端未对接	3 个（可立即推进）

第三步：确定优先级

- 最快出成果：对接 3 个 P1 功能（后端已写好，前端换掉 Mock 即可）→ 见第 7 章
- 完整任务清单：68 个任务、预估 134 小时 → 见附录 B
- 每个功能的详细状态、API、代码路径 → 见附录 A

第四步：启动后端（需要时再做）

```
cd ai-hms-backend && cp .env.example .env
# 编辑 .env 填入 PostgreSQL 连接信息
psql -U postgres -c "CREATE DATABASE ai_hms_db;"
go run cmd/server/main.go

登录账号: admin / admin123 (硬编码测试账号)
```

第 1 章 项目概述

AI-HMS 是一套面向透析中心的医院信息管理系统，覆盖患者管理、透析治疗流程、设备监控、排班管理、统计分析等核心业务。

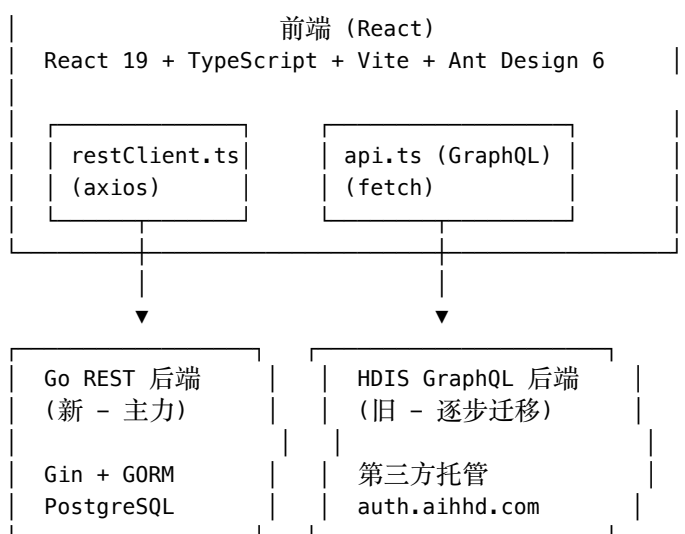
项目	说明
系统类型	透析中心 HIS
前端技术栈	React 19 + TypeScript 5.9 + Vite 7 + TailwindCSS v4 + Ant Design 6
后端技术栈	Go 1.24 + Gin + GORM + PostgreSQL
旧后端	HDIS GraphQL API（第三方，部分接口仍在使用）
国际化	中文 / 英文 (i18next)
页面数量	16 个路由页面
功能模块	46 个功能点
前端代码量	~33,000 行
后端代码量	~11,700 行
后端活跃路由	96 个
数据库模型	35+ 张表

核心业务模块

模块	页面	说明
仪表板	Dashboard	透析中心全局看板
患者管理	PatientList, PatientDetail	患者信息、病史、血管通路、治疗转归
透析治疗	DialysisProcessing	透析执行流程（透前/透中/透后）
设备监控	Monitoring, DeviceManagement	实时监控、设备绑定管理
排班管理	Schedule	患者排班、床位班次
病区总览	WardOverview	病区/床位实时状态
统计分析	Statistics	数据统计报表
治疗配置	TreatmentConfig	方案模板、医嘱模板
字典配置	DictConfig	系统字典类型与字典项管理
主数据管理	MasterData	药品目录、材料目录
耗材管理	Inventory	耗材库存管理
系统设置	Settings, Login, RoleSelect	登录、角色选择、API 配置

第 2 章 技术架构





系统采用双 API 架构，正在从旧的 GraphQL 后端迁移到新的 Go REST 后端。

数据源优先级：REST API → GraphQL API → Mock 数据（本地硬编码）

当 REST API 未配置或不可用时，系统会自动 Fallback 到 Mock 数据，保证前端可以独立运行和演示。

第 3 章 环境搭建

3.1 前置要求

工具	版本要求
Node.js	>= 18
npm	>= 9
Go	>= 1.24
PostgreSQL	>= 14
Chrome/Chromium	需要（HDIS Token 自动获取依赖 chromedp）

3.2 前端

```

cd ai-hms-frontend
npm install
npm run dev      # 开发服务器 → http://localhost:5173
npm run build    # 生产构建 (tsc -b && vite build)
npm run lint     # ESLint 9
  
```

环境变量文件：

文件	用途	关键变量
.env	通用	VITE_BACKEND_API_URL=http://localhost:8080
.env.development	开发	VITE_API_BASE_URL=http://localhost:8080
.env.production	生产	VITE_API_BASE_URL=http://47.239.21.188:8080

即使不启动后端，前端也可以独立运行（自动 Fallback 到 Mock 数据）。

3.3 后端

```
cd ai-hms-backend
cp .env.example .env      # 复制配置模板
# 编辑 .env, 填入实际数据库连接信息

psql -U postgres -c "CREATE DATABASE ai_hms_db;" # 创建数据库
go run cmd/server/main.go                       # 启动 -> http://localhost:8080
```

后端.env 关键配置:

```
# 服务器
SERVER_PORT=8080
GIN_MODE=debug          # debug 模式下自动执行数据库迁移

# 数据库
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD= 你的密码
DB_NAME=ai_hms_db
DB_SSL_MODE=disable

# JWT
JWT_SECRET= 你的密钥
JWT_EXPIRATION_HOURS=24

# CORS
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000

# HDIS 集成 (可选, 对接第三方 HDIS 系统时才需要)
HDIS_AUTH_URL=https://auth.aihd.com/connect/token
HDIS_BROWSER_HEADLESS=true
HDIS_BROWSER_TIMEOUT_SECONDS=120
HDIS_TARGET_ORGAN_ID=2
```

首次启动时后端会自动执行: 1. 数据库表自动迁移 (AutoMigrate, 仅 GIN_MODE=debug 时) 2. 种子数据初始化 (25 个字典类型、~120 个字典项) 3. 启动过期医嘱定时任务

3.4 测试登录

账号	密码	说明
admin	admin123	硬编码测试账号, 后续需替换为真实用户体系

第 4 章 项目结构

4.1 前端

```
ai-hms-frontend/src/
├── components/      # 可复用组件
│   ├── ui/         # 基础 UI 组件
│   └── patient/     # 患者业务组件 (Modal 等)
├── layouts/         # 布局组件 (MainLayout, Sidebar, Header)
└── pages/           # 16 个路由页面
```

```

├── patient-detail/tabs/ # 患者详情 9 个 Tab 页
├── services/           # API 服务层（核心，见第 5 章）
│   ├── restClient.ts  # ★ REST API 客户端（主力）
│   ├── api.ts         # GraphQL 客户端（旧，逐步废弃）
│   ├── dictApi.ts     # 字典服务
│   ├── orderApi.ts    # 医嘱服务
│   ├── patientApi.ts  # 患者服务
│   └── treatmentConfigApi.ts # 治疗配置服务
├── hooks/             # 自定义 Hooks (useDictName, useOutcomeDict)
├── contexts/          # React Context (Auth, Theme)
├── i18n/locales/      # 国际化翻译文件 (zh-CN, en-US)
├── types/             # TypeScript 类型定义
├── utils/             # 工具函数
├── constants.ts       # ★ Mock 数据源（对接真实 API 后逐步删除）
└── router.tsx         # 路由配置

```

4.2 后端

```

ai-hms-backend/
├── cmd/server/main.go # ★ 入口（路由注册、中间件、种子数据）
├── config/config.go   # 环境变量配置加载
├── internal/
│   ├── api/v1/        # HTTP Handler（18 个文件）
│   ├── database/      # 数据库连接 + 自动迁移
│   ├── integrations/hdis/ # HDIS 集成（浏览器自动登录、GraphQL 客户端）
│   ├── middleware/    # JWT 认证、CORS
│   ├── models/        # GORM 数据模型（11 个文件，42 个结构体）
│   ├── services/      # 业务逻辑层（29 个文件）
│   └── utils/         # JWT 管理等工具
├── pkg/response/      # 标准 HTTP 响应封装
└── logs/              # 运行时日志 (app.log / error.log)

```

后端代码分层：新功能开发遵循 Model → Service → Handler → 路由注册 的流水线模式，参考任何已有模块即可。

第 5 章 双 API 架构说明

5.1 REST API (Go 新后端—主力)

项目	说明
客户端	src/services/restClient.ts (axios 实例)
Base URL	环境变量 VITE_API_BASE_URL，默认 http://localhost:8080
认证	Bearer Token (localStorage: hdis_access_token)
超时	10 秒
401 处理	自动清除 Token 并跳转登录页

已覆盖模块：

- 用户登录认证
- 患者管理（CRUD + 详情 + 基本信息 + 核心聚合）
- 病史与转归记录、血管通路 + 干预记录
- 治疗方案 + 调整记录、医嘱管理 + 处方管理
- 检验报告 / 检查报告 / 关键指标（含 HDIS 同步）

- 字典配置、治疗配置（方案模板 / 医嘱模板 / 药品目录 / 材料目录）
- 排班管理（后端已实现，前端未对接）
- HDIS 集成设置、系统日志

5.2 GraphQL API（HDIS 旧后端—待迁移）

项目	说明
客户端	src/services/api.ts (fetch + FormData)
API 地址	用户配置的 tenantAddress + /pygql
认证	hdis_access_token
特殊点	HDIS 使用非标准 FormData 传参，参数需要双重 JSON 转义

仅剩 3 个模块仍在使用：

功能	页面	说明
实时运行指标	Dashboard	今日透析/设备/班次
设备列表 + 消毒记录	设备管理	EquipmentInfomation/EquipmentDisinfection
病区指标	病区总览	排班/设备利用率部分

5.3 Mock 数据（Fallback）

- 数据源：src/constants.ts + 各页面内硬编码常量
- 触发条件：REST API 未配置或请求失败时自动 Fallback
- 覆盖范围：24 个功能模块仍使用 Mock 数据
- 对接真实 API 后，逐步删除 constants.ts 中对应的 Mock 数据即可

第 6 章 数据库说明

6.1 基本信息

项目	说明
数据库	PostgreSQL
ORM	GORM 1.31
迁移方式	AutoMigrate（GIN_MODE=debug 时自动执行）
模型文件	internal/models/ 下 11 个文件
表数量	35+ 张表
外键约束	迁移时禁用（DisableForeignKeyConstraintWhenMigrating: true）

6.2 核心模型文件

文件	包含的表	说明
patient.go	患者、血管通路、干预记录、病史、转归记录、感染信息	患者域核心

文件	包含的表	说明
patient_basic_info.go	患者基本信息扩展	身份/医疗/社会信息等 6 大块
treatment.go	治疗方案、透析参数、抗凝剂、材料、处方、医嘱、治疗记录 (透前/透中/透后)、报警	最大的模型文件
treatment_config.go	方案模板、材料目录、药品目录、医嘱模板	治疗配置域
schedule.go	病区、床位、班次、患者排班	排班域
dict.go	字典类型、字典项	多个模块的上游依赖
user.go	用户	当前仅硬编码账号
hospitalization.go	住院记录	
lab_report.go	检验报告、检验项目、检查报告	含 HDIS 同步
patient_key_indicator.go	患者关键指标	HGB/Ca/P 等
integration_hdis_setting.go	HDIS 集成配置	存储第三方连接信息

6.3 ID 策略

系统中存在两种 ID 生成策略，这是一个已知的历史问题，跨域关联时需要类型转换：

策略	类型	使用的表
UUID	varchar(36)	patients, treatment_plans, vascular_accesses, users, dicts 等
自增	bigint	treatment_*, wards, beds, shifts, patient_shifts, catalogs 等

特别注意：Patient.ID 是 string (UUID)，而 Treatment.PatientId 是 int64。跨域查询时需要类型转换层。

第 7 章 功能完成度与后续工作量

7.1 整体进度

状态	含义	数量	占比
Done	前后端完整对接，功能可用	18	39%
Partial	部分对接，部分仍为 Mock 或硬编码	4	8%
Mock	前端已有 UI，但数据来自假数据	24	52%
总计		46	100%

每个功能的详细状态、对应 API、前后端代码路径，见[附录 A](#)。

7.2 按页面完成度

页面	功能数	Done	Mock	完成度
患者详情 (/patients/:id)	9	7	2	78%
治疗配置 (/treatment-config)	4	4	0	100%
系统设置 (/settings)	3	2	0	67%
透析流程 (/dialysis-processing)	6	1	5	17%
Dashboard (/dashboard)	5	1	3	20%
统计分析 (/statistics)	4	0	4	0%
设备监控 (/monitoring)	2	0	2	0%
排班管理 (/schedule)	2	0	2	0%
耗材管理 (/inventory)	2	0	2	0%

7.3 下一步该做什么（按优先级）

P1: 后端已就绪，只需前端对接（3个）

这3个功能的后端API已经写好了，前端只需要把Mock数据替换成真实API调用。最快出成果的切入点。

功能	页面	后端API	当前状态
科室患者列表	Dashboard	GET /api/v1/patients	前端通过 mockHelpers.ts 临时实现，换成 restApi.getPatientList() 即可
透前评估	透析流程	GET/POST /api/v1/treatments	前端用 MOCK_PATIENTS + MOCK_SESSION_PROCESS，后端 CRUD 已就绪
患者排班	排班管理	shifts + patient-shifts 共8个端点	高优先级：后端完整 CRUD 含冲突检查，前端仍用 MOCK_PATIENT_SCHEDULE

P2: 需从 GraphQL 迁移到 REST（3个）

这3个功能目前走旧的 HDIS GraphQL 后端，需要在新 Go 后端实现对应API，然后前端切换数据源。

功能	页面	说明
实时运行指标	Dashboard	getTodayTreatments/getAllEquipments/getActiveShifts 走 GraphQL
设备列表 + 消毒记录	设备管理	EquipmentInfomation/EquipmentDisinfection 走 GraphQL
病区指标	病区总览	排班/设备利用率走 GraphQL，患者总数已走 REST

P3: 需要全栈开发 (20 个)

需要后端建模 + API 实现 + 前端对接的完整开发工作。包括：透析核心流程（核对/监测/处方/透后评估）、设备监控、护士排班、统计分析全部 4 个 Tab、耗材管理、库存管理、主数据管理等。

完整 20 个功能的详细说明见附录 A 的 P3 部分。

7.4 后续工作量估算

阶段	任务数	预估工时	核心目标
Phase 1: 患者后端闭环	15	31h	补齐缺失 API + restClient 封装
Phase 2: 患者前端 + 透析后端	16	32h	8 个 Tab 脱 Mock + 透析子流程后端
Phase 3: 透析前端 + 设备全栈	15	34h	9 Tab 闭环 + 透析全流程 + 设备
Phase 4: 发布门禁	22	37h	认证去硬编码 + 安全 + GraphQL 退役
合计	68	134h	

按每天 6-8 小时全职开发计算，约需 4 周（20 个工作日）。详细的 68 个任务明细、依赖关系和风险管理见[附录 B](#)。

第 8 章 已知问题与注意事项

8.1 测试服务器 vs 医院线上系统存在差异

开发过程中前端基于测试服务器的 API 和数据开发，但验收时发现线上系统的部分接口行为与测试服不一致，导致多次返工。

接手时务必注意：- 对接 HDIS 接口前，先在线上环境验证接口返回格式 - 不要完全信任测试服的 API 响应，以线上为准

8.2 HDIS 集成特殊性

- HDIS Token 获取依赖 **chromedp** (headless Chrome 自动化登录)，相关代码在 `internal/integrations/hdis/`
- Token 有效期有限，需要定期刷新
- HDIS GraphQL 接口使用非标准的 FormData 传参，参数需要双重 JSON 转义

8.3 登录认证为硬编码

- 当前为硬编码测试账号 (admin/admin123)，位于 `cmd/server/main.go:147-155`
- 后续需实现真实用户认证 (bcrypt + JWT)，并删除硬编码
- JWT Token 有效期 24 小时

8.4 数据库注意事项

- AutoMigrate 仅在 GIN_MODE=debug 时执行，生产环境需要手动管理迁移
- 外键约束在迁移时是禁用的
- 主键类型不一致 (UUID vs 自增 bigint)，跨域关联需要类型转换（见第 6.3 节）

8.5 前端 UI 变动遗留

- 项目过程中 UI 设计稿经历了多次变更，部分页面可能存在样式不一致的情况
- TailwindCSS v4 和 Ant Design 6 两套样式体系并存

8.6 高风险问题清单 (P0)

问题	位置	说明
登录硬编码 admin/admin123	cmd/server/main.go:147-155	必须在线上前替换
患者创建缺事务保护	patient_service.go:180-287	中途失败会留残数据
Preload 关联名错误	patient_service.go:117,378	VascularAccess → VascularAccesses
TenantId/CreatorId 硬编码为 1	4 个 service 共 8 处	需实现多租户中间件

第 9 章 后续开发建议

9.1 推荐的开发顺序

1. 先做 P1: 3 个功能后端已就绪，前端换掉 Mock 就能用，最快见效
2. 再做 P2: 3 个 GraphQL → REST 迁移
3. 然后做 P3: 20 个全栈开发任务
4. 最后做发布门禁：认证、安全、GraphQL 完全退役

9.2 技术要点

- 新增后端功能：参考已有模块，按 Model → Service → Handler → main.go 路由注册 的模式
- 新增前端 API 对接：在 restClient.ts 的 RestApiService 类中添加方法，参考已有方法
- Mock 数据清理：对接真实 API 后删除 constants.ts 中对应的数据
- 字典系统：DictConfig 是多个模块的上游依赖（治疗配置、患者管理等），修改字典时注意影响范围

9.3 AI 辅助开发

项目根目录配置了 .claude/ 目录，包含完整的项目上下文和开发规范：

文件	用途
.claude/CLAUDE.md	项目概览、常用命令、架构说明
.claude/skills/coding-standard.md	React/TS 编码规范
.claude/skills/component-design.md	组件设计规范
.claude/skills/api-contract.md	双 API 契约 + Mock 管理

文件	用途
.claude/skills/database-design.md	数据库设计规范
.claude/skills/i18n.md	国际化维护指南

配合 AI 编码工具（Claude Code、Cursor 等）使用时，AI 会自动读取这些文件来理解项目上下文。

9.4 角色权限参考

角色代码	角色名称
DOCTOR_CHIEF	主任医师
DOCTOR_SUPERVISOR	主治医师
DOCTOR_DUTY	值班医师
NURSE_HEAD	护士长
NURSE_MANAGER	护理组长
NURSE_RESPONSIBLE	责任护士
ENGINEER	工程师
ADMIN	系统管理员

附录 A 产品功能地图

本附录是 46 个功能的完整状态参考。每个功能标注了当前状态、数据源、后端是否就绪、对应的 API 端点、前端代码路径和后端代码路径。

状态定义：Done = 前后端已对接可用 | Partial = 部分对接 | Mock = 仅前端 UI + 假数据

数据源：REST = Go 新后端 | GraphQL = HDIS 旧后端 | Mock = 前端硬编码 | Local = 浏览器本地

后端就绪 (Y)：后端 API 已实现，即使前端尚未对接

A.1 路由级总览

路由	页面文件	功能数	Done	Mock	数据源
/dashboard	src/pages/Dashboard.tsx	5	1	3	GraphQL / Local / Mock
/device-binding	src/pages/DeviceManagement.tsx	2	0	1	GraphQL / Mock
/dialysis-processing	src/pages/DialysisProcessing.tsx	6	1	5	Local / Mock
/dict-config	src/pages/DictConfig.tsx	1	1	0	REST
/inventory	src/pages/Inventory.tsx	2	0	2	Mock
/login	src/pages/Login.tsx	1	1	0	REST

路由	页面文件	功能数	Done	Mock	数据源
/master-data	src/pages/MasterData.tsx	1	0	1	Mock
/monitoring	src/pages/Monitoring.tsx	2	0	2	Mock
/patients	src/pages/PatientList.tsx	1	1	0	REST
/patients/:id	src/pages/PatientDetail.tsx	9	7	2	Mock / REST
/role-select	src/pages/RoleSelect.tsx	1	0	1	Mock
/schedule	src/pages/Schedule.tsx	2	0	2	Mock
/settings	src/pages/Settings.tsx	3	2	0	Local / REST
/statistics	src/pages/Statistics.tsx	4	0	4	Mock
/treatment-config	src/pages/TreatmentConfig.tsx	4	4	0	REST
/ward-overview	src/pages/WardOverview.tsx	2	0	1	GraphQL / Mock

A.2 按页面详细功能

/dashboard

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
DASH-ALERTS-001	待处理处方/医嘱提示	Mock	Mock	-	-	src/pages/Dashboard.tsx	-
DASH-LAYOUT-001	可定制化卡片布局 + 角色权限过滤	Done	Local	-	-	src/pages/Dashboard.tsx	-
DASH-METRICS-001	实时运行指标(今日透析/设备/班次)	Partial	GraphQL	-	-	src/pages/Dashboard.tsx src/services/treatment.ts src/services/equipment.ts src/services/schedule.ts	-
DASH-PATIENTS-001	科室患者数 + 我的责任患者列表	Mock	Mock	Y	GET /api/v1/patients	src/pages/Dashboard.tsx src/utils/mockHelpers.ts	internal/api/v1/patient_handler.go
DASH-STATS-001	质量达标率/护士工作量图表	Mock	Mock	-	-	src/pages/Dashboard.tsx src/constants.ts	-

- DASH-ALERTS-001: 硬编码示例数据(低血压报警、医嘱请求)。需设计告警 API。

- DASH-LAYOUT-001: 布局配置存 localStorage, 卡片按 card.roles 过滤。纯前端功能, 无需后端。
- DASH-METRICS-001: 三个 GraphQL 调用需迁移到 REST。
- DASH-PATIENTS-001: P1 可立即推进。后端 REST API 已就绪, 前端换成 restApi.getPatientList() 即可。
- DASH-STATS-001: 硬编码数据。需设计统计报表 API。

/dialysis-processing

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
DIALYSIS-CHECK-001	首次核对 (6 项) + 二次核对 (5 项)	Mock	Mock	-	-	src/pages/DialysisProcessing.tsx	-
DIALYSIS-MON-001	透中监测 (生命体征/设备参数记录)	Mock	Mock	-	-	src/pages/DialysisProcessing.tsx src/constants.ts	-
DIALYSIS-POST-001	透后评估 + 透析小结	Mock	Mock	-	-	src/pages/DialysisProcessing.tsx	-
DIALYSIS-PRE-001	透前评估 (体重/血压/症状/穿刺)	Mock	Mock	Y	GET/POST /api/v1/treatments	src/pages/DialysisProcessing.tsx src/constants.ts	internal/api/v1/treatment_handler.go
DIALYSIS-PRINT-001	A4 透析单打印预览	Done	Local	-	-	src/pages/DialysisProcessing.tsx	-
DIALYSIS-RX-001	当日处方 (透析参数/抗凝/耗材)	Mock	Mock	-	-	src/pages/DialysisProcessing.tsx src/constants.ts	-

- DIALYSIS-CHECK-001: 静态核对清单, 本地 checkbox 状态。需设计核对记录 API。
- DIALYSIS-MON-001: 监测记录表格支持 CRUD, 含 16+ 参数输入。“获取设备数据”按钮待对接实时设备。
- DIALYSIS-POST-001: 透后体重/血压/穿刺点状态/并发症等字段。含医嘱、设备消毒、健康宣教占位步骤。
- DIALYSIS-PRE-001: P1 可立即推进。后端 treatment CRUD 已就绪但前端未对接。
- DIALYSIS-PRINT-001: PrintView 组件, 完整 A4 报告格式。数据来源于当前页面状态。
- DIALYSIS-RX-001: 处方数据来自患者默认方案常量。需与治疗方案 API 联动。

/patients

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
PATIENT-001	患者列表 CRUD + 搜索筛选	Done	REST	Y	GET /api/v1/patients POST /api/v1/patients DELETE /api/v1/patients/{id}	src/pages/PatientList.tsx src/services/restClient.ts src/components/patient/CreatePatientModal.tsx src/hooks/useDictName.ts	internal/api/v1/patient_handler.go internal/services/patient_service.go

/patients/:id

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
PAT-BASIC-001	基本信息 Tab	Done	REST	Y	GET/PUT /api/v1/patients/{id}/basic-info	src/pages/patient-detail/tabs/BasicInfoTab.tsx	internal/api/v1/patient_basic_handler.go internal/services/patient_basic_service.go
PAT-CORE-001	患者详情 头部 + 核心聚合	Done	REST	Y	GET /api/v1/patients/{id}/core	src/pages/PatientDetail.tsx	internal/api/v1/patient_core_handler.go internal/services/patient_core_service.go
PAT-HISTORY-001	治疗历史 Tab	Mock	Mock	-	-	src/pages/patient-detail/tabs/HistoryTab.tsx	-
PAT-LABS-001	检验/检查/关键指标 Tab	Done	REST	Y	GET/POST /api/v1/patients/{id}/lab-reports GET/POST /api/v1/patients/{id}/exam-reports 等	src/pages/patient-detail/tabs/LabsExamsTab.tsx	internal/api/v1/lab_report_handler.go 等
PAT-MEDICAL-001	病史与转归 Tab	Done	REST	Y	GET/PUT /api/v1/patients/{id}/medical-history CRUD /api/v1/patients/{id}/outcome-records	src/pages/patient-detail/tabs/MedicalRecordTab.tsx	internal/api/v1/medical_history_handler.go
PAT-MONTHLY-001	月度总结 Tab	Mock	Mock	-	-	src/pages/patient-detail/tabs/MonthlySummaryTab.tsx	-
PAT-PLAN-001	治疗方案 Tab	Done	REST	Y	CRUD /api/v1/patients/{id}/treatment-plan	src/pages/patient-detail/tabs/TreatmentPlanTab.tsx	internal/api/v1/patient_handler.go
PAT-SCHEME-001	医嘱方案 Tab	Done	REST	Y	16 个端点 (orders + prescriptions 完整 CRUD)	src/pages/patient-detail/tabs/SchemeOrderTab.tsx src/services/orderApi.ts	internal/api/v1/order_handler.go internal/api/v1/prescription_handler.go
PAT-VASCULAR-001	血管通路 + 干预记录 Tab	Done	REST	Y	CRUD /api/v1/patients/{id}/vascular-accesses CRUD ../vascular-access-interventions	src/pages/patient-detail/tabs/VascularTab.tsx	internal/api/v1/vascular_access_handler.go

- PAT-CORE-001: 首屏接口返回 header + overview + clinicalFocus。路由必须在 /:id 之前注册。
- PAT-HISTORY-001: P3, 需设计治疗历史查询 API。
- PAT-LABS-001: 最常用排查链路。含 LIS/检查同步功能。
- PAT-MONTHLY-001: P3, 纯前端占位表单, 需设计月度汇总聚合 API。
- PAT-SCHEME-001: 最复杂的 Tab, 含长期/临时医嘱、模板导入、组合/拆组、修订、复制、历史库、医嘱单。

/schedule

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
SCHED-PATIENT-001	患者排班	Mock	Mock	Y	shifts + patient-shifts 共 8 个 CRUD 端点	src/pages/Schedule.tsx	internal/api/v1/schedule_handler.go
SCHED-STAFF-001	护士排班	Mock	Mock	-	-	src/pages/Schedule.tsx	-

- SCHED-PATIENT-001: P1 高优先级。后端完整 CRUD 含冲突检查已实现，前端仍用 Mock。
- SCHED-STAFF-001: P3，需设计护士排班后端 API。仅护士角色可见。

/treatment-config

ID	功能	状态	数据源	后端就绪	API	前端代码	后端代码
TC-DRUG-001	药品目录管理 (CRUD/Excel 导入导出)	Done	REST	Y	CRUD /api/v1/drugs/catalog	src/pages/TreatmentConfig.tsx src/services/treatmentConfigApi.ts	internal/api/v1/treatment_config_handler.go
TC-MATERIAL-001	材料目录管理 (CRUD/Excel 导入导出)	Done	REST	Y	CRUD /api/v1/materials/catalog	同上	同上
TC-ORDER-001	医嘱模板管理 (CRUD/药品搜索)	Done	REST	Y	CRUD /api/v1/order-templates	同上	同上
TC-PLAN-001	方案模板管理 (CRUD/启用/默认)	Done	REST	Y	CRUD /api/v1/treatment-templates	同上	同上

- 治疗配置 4 个子模块全部完成，是系统中完成度最高的页面。
- TC-PLAN-001 依赖 dictCache 加载 13 种字典类型（5 分钟 TTL）。

其他页面

ID	页面	功能	状态	说明
DICT-001	/dict-config	字典类型 CRUD + 字典项树形管理	Done	多个页面上游配置中心。含批量导入和本地 JSON 导出。
AUTH-LOGIN-001	/login	用户名密码登录	Done	后端使用内置测试账号，后续应替换为真实用户体系。
AUTH-ROLE-001	/role-select	角色用户列表与选择	Mock	8 人硬编码。需对接用户管理 API。
SET-HDIS-001	/settings	HDIS 集成配置 + Token 刷新	Done	WebCmd/GraphQL/Auth URL + 自动刷新配置。

ID	页面	功能	状态	说明
SET-LOGS-001	/settings	系统日志查询	Done	10 秒自动刷新。支持 source/level/keyword/lines 查询。
SET-DISPLAY-001	/settings	显示设置 (主题/语言)	Partial	主题/语言已完成, 账户/通知/安全 Tab 为 Mock 占位。
DEV-LIST-001	/device-binding	设备列表 + 详情 + 消毒记录	Partial	走 HDIS GraphQL, 需迁移到 REST。
DEV-STATUS-001	/device-binding	设备状态 + 统计卡片	Mock	设备状态由 ID 模 8 余数模拟。
MON-PANEL-001	/monitoring	设备监控面板 (25 台设备)	Mock	含 Mini 生命体征图表。需 WebSocket/轮询方案。
MON-DETAIL-001	/monitoring	综合透中监测弹窗	Mock	详细参数展示 (血压趋势/压力/流量等)。
INV-STOCK-001	/inventory	实时库存 Tab	Mock	硬编码 6 项耗材。需设计库存管理 API。
INV-LOG-001	/inventory	出入库流水 Tab	Mock	5 条流水记录硬编码。
MASTER-DATA-001	/master-data	主数据管理	Mock	纯内存操作, 刷新丢失。与 TreatmentConfig 部分功能重叠。
WARD-BED-001	/ward-overview	床位状态矩阵	Mock	32 张床位随机状态模拟。需实时床位状态 API。
WARD-METRICS-001	/ward-overview	病区指标	Partial	患者总数走 REST, 排班/设备走 GraphQL。
STAT-QUALITY-001	/statistics	质量指标 Tab	Mock	6 个月硬编码数据。需设计质量指标聚合 API。
STAT-INFECTION-001	/statistics	感染标志物 Tab	Mock	需从患者传染病数据聚合。
STAT-VASCULAR-001	/statistics	血管通路 Tab	Mock	可从血管通路数据聚合。
STAT-WORKLOAD-001	/statistics	工作量统计 Tab	Mock	5 名护士硬编码表格。

附录 B 后续开发计划

本附录提供后续 68 个开发任务的完整明细, 供接手者参考排期。

原始基线: 每周 5 天、每天 6-8 小时、全职 AI 辅助开发、4 周完成。接手者应根据自身情况重新评估工时。

B.1 阶段划分

Phase 1 — 患者后端闭环	15 任务	31h	纯后端, 补齐缺失 API
Phase 2 — 患者前端+透析	16 任务	32h	8 Tab 脱 Mock + 透析后端
Phase 3 — 透析前端+设备	15 任务	34h	9 Tab 闭环 + 设备全栈
Phase 4 — 发布门禁	22 任务	37h	认证 + 安全 + GraphQL 退役
合计	68 任务	134h	20 工作日

B.2 后端模块缺口

以下是目前尚未创建的后端文件，按开发阶段排列：

缺失文件	模块	阶段
models/lab_report.go	患者管理	P1
services/lab_report_service.go	患者管理	P1
api/v1/lab_report_handler.go	患者管理	P1
services/order_service.go	医嘱管理	P1
api/v1/order_handler.go	医嘱管理	P1
models/monthly_summary.go	患者管理	P1
services/monthly_summary_service.go	患者管理	P1
api/v1/monthly_summary_handler.go	患者管理	P1
services/treatment_sub_service.go	透析治疗	P2
models/equipment.go	设备监控	P3
services/equipment_service.go	设备监控	P3
api/v1/equipment_handler.go	设备监控	P3
models/inventory.go	耗材管理	P4
services/inventory_service.go	耗材管理	P4
api/v1/inventory_handler.go	耗材管理	P4
services/statistics_service.go	仪表板	P4
api/v1/statistics_handler.go	仪表板	P4

B.3 全量任务表（68 条）

ID	阶段	模块	任务	涉及文件	预估
1	P1	患者管理	LabReport 数据模型 + 注册 migrate.go	models/lab_report.go(新), migrate.go	2h
2	P1	患者管理	lab_report_service.go CRUD 实现	services/lab_report_service.go(新)	3h
3	P1	患者管理	lab_report_handler.go + 4 端点路由注册	api/v1/lab_report_handler.go(新), main.go	2h
4	P1	患者管理	restClient.ts 封装检验报告 API (4 端点)	restClient.ts	2h
5	P1	医嘱管理	order_service.go CRUD + 状态变更	services/order_service.go(新)	2h
6	P1	医嘱管理	order_handler.go + 5 端点路由注册	api/v1/order_handler.go(新), main.go	2h
7	P1	医嘱管理	Order 5 端点联调 + restClient.ts 封装	restClient.ts	2.5h
8	P1	患者管理	MonthlySummary 数据模型 + 注册 migrate.go	models/monthly_summary.go(新), migrate.go	2h
9	P1	患者管理	monthly_summary_service.go CRUD	services/monthly_summary_service.go(新)	2.5h
10	P1	患者管理	monthly_summary_handler.go + 4 端点路由注册	api/v1/monthly_summary_handler.go(新), main.go	2h
11	P1	患者管理	restClient.ts 封装月度小结 API (4 端点)	restClient.ts	1.5h
12	P1	患者管理	/core buildActiveOrders 真实化	patient_core_service.go:165	2.5h
13	P1	患者管理	/core buildLabTrends 真实化	patient_core_service.go:188	2h

ID	阶段	模块	任务	涉及文件	预估
14	P1	患者管理	/core buildClinicalFocus 真实化	patient_core_service.go:204	2h
15	P1	验收	P1 验收: 13 个新端点 + restClient + /core 非空数据	—	1h
16	P2	患者管理	Preload 关联名修复	patient_service.go:117,378	1h
17	P2	患者管理	患者创建事务化 (db.Transaction)	patient_service.go:180-287	2h
18	P2	患者管理	LabsExamsTab.tsx 接入检验报告 API	LabsExamsTab.tsx	2h
19	P2	患者管理	SchemeOrderTab.tsx 接入医嘱 API	SchemeOrderTab.tsx, constants.ts	2h
20	P2	患者管理	MonthlySummaryTab.tsx 接入月度小结 API	MonthlySummaryTab.tsx	2h
21	P2	患者管理	OverviewTab.tsx 趋势图接入 /core API	OverviewTab.tsx:67-104	2h
22	P2	患者管理	患者详情 8 Tab 联调 + 空态处理	PatientDetail tabs	2h
23	P2	透析治疗	treatment_sub_service.go 骨架 + BeforeCheck CRUD	treatment_sub_service.go(新)	2.5h
24	P2	透析治疗	BeforeSigns CRUD (Service + Handler)	treatment_sub_service.go, treatment_handler.go	2h
25	P2	透析治疗	DuringParam CRUD (Service + Handler)	treatment_sub_service.go, treatment_handler.go	2.5h
26	P2	透析治疗	AfterSigns CRUD (Service + Handler)	treatment_sub_service.go, treatment_handler.go	2h
27	P2	透析治疗	Alarm CRUD (Service + Handler)	treatment_sub_service.go, treatment_handler.go	2h
28	P2	透析治疗	Patient.ID ↔ Treatment.PatientId 类型转换层	treatment_service.go	2h
29	P2	透析治疗	治疗历史 API (GET /patients/:id/treatment-history)	treatment_handler.go, treatment_service.go	2h
30	P2	透析治疗	/core 治疗会话状态判断实现	patient_core_service.go:65	2h
31	P2	透析治疗	restClient.ts 封装透析治疗 API	restClient.ts	2h
32	P3	透析治疗	HistoryTab.tsx 接入治疗历史 API → 患者 9Tab 闭环	HistoryTab.tsx	2h
33	P3	透析治疗	DialysisProcessing.tsx Mock 依赖分析 + 接入患者列表	DialysisProcessing.tsx	3h
34	P3	透析治疗	接入透前检查/体征 API	DialysisProcessing.tsx	2h
35	P3	透析治疗	接入透中监测参数 API	DialysisProcessing.tsx	3h
36	P3	透析治疗	接入透后体征 + 报警 API	DialysisProcessing.tsx	2h
37	P3	排班管理	排班 API 最小对接 (透析依赖的排班查询)	DialysisProcessing.tsx, schedule_handler.go	2h
38	P3	透析治疗	透前 → 透中 → 透后全流程联调	DialysisProcessing.tsx	2.5h
39	P3	设备监控	Equipment 数据模型 + 注册 migrate.go	models/equipment.go(新), migrate.go	2h
40	P3	设备监控	equipment_service.go CRUD	services/equipment_service.go(新)	2.5h
41	P3	设备监控	equipment_handler.go + 6 端点路由注册	api/v1/equipment_handler.go(新), main.go	3h
42	P3	设备监控	restClient.ts 封装设备 API	restClient.ts	2h
43	P3	设备监控	设备端点联调验证	—	1h
44	P3	设备监控	Monitoring.tsx 接入设备 REST API	Monitoring.tsx	3h
45	P3	设备监控	DeviceManagement.tsx 从 GraphQL 切 REST	DeviceManagement.tsx	3h

ID	阶段	模块	任务	涉及文件	预估
46	P3	验收	P3 验收: 9Tab + 透析全流程 + 设备 + npm run build	—	1h
47	P4	认证	user_auth_service (bcrypt + JWT)	user_auth_service.go(新), user.go	3h
48	P4	认证	删除 main.go 硬编码登录 + 联调	main.go:147-155	1.5h
49	P4	多租户	多租户中间件 TenantId/CreatorId 提取	auth.go, treatment_service.go	2.5h
50	P4	多租户	更新 4 个 service 中 8 处硬编码	shift_service.go, patient_shift_service.go, hospitalization_service.go, treatment_service.go	2h
51	P4	认证	基础角色门禁中间件	auth.go, main.go	1.5h
52	P4	工程化	API 请求参数校验中间件	middleware/validation.go(新)	1.5h
53	P4	工程化	关键操作审计日志	middleware/audit_log.go(新)	1h
54	P4	耗材管理	Inventory 数据模型 + 注册 migrate.go	models/inventory.go(新), migrate.go	2h
55	P4	耗材管理	inventory_service.go CRUD	services/inventory_service.go(新)	2h
56	P4	耗材管理	inventory_handler.go + 5 端点路由注册	api/v1/inventory_handler.go(新), main.go	2h
57	P4	排班/耗材	restClient.ts 封装排班/耗材 API	restClient.ts	1.5h
58	P4	耗材管理	Inventory.tsx 去 Mock + 联调	Inventory.tsx	1.5h
59	P4	排班管理	Schedule.tsx 接入排班 REST API (11 个已有端点)	Schedule.tsx	3h
60	P4	仪表板	statistics_service.go + handler + 统计端点	statistics_service.go(新), statistics_handler.go(新)	2.5h
61	P4	仪表板	Dashboard.tsx + WardOverview.tsx 接入统计/切 REST	Dashboard.tsx, WardOverview.tsx	2.5h
62	P4	架构	删除零消费 GraphQL 服务	services/vitals.ts, order.ts, examination.ts	1h
63	P4	架构	删除已迁移 GraphQL 服务 + api.ts + npm run build	services/schedule.ts, treatment.ts, equipment.ts, api.ts	1.5h
64	P4	工程化	后端 Go test: 认证 + 患者 CRUD	*_test.go(新)	1.5h
65	P4	工程化	前端 Vitest: PatientList + PatientDetail Tab 切换	vitest.config.ts, *.test.tsx(新)	1h
66	P4	工程化	golang-migrate 引入 + 初始迁移脚本	migrations/(新)	1h
67	P4	验收	发布检查清单编写	—	0.5h
68	P4	验收	最终回归测试	—	0.5h

B.4 工时统计

按阶段

阶段	任务数	总工时	核心目标
P1 患者后端 闭环	15	31h	补齐缺失 API + restClient 封装 + /core 真实化
P2 患者前端 + 透析后端	16	32h	8 Tab 脱 Mock + 透析子流程后端
P3 透析前端 + 设备全栈	15	34h	9 Tab 完整 + 透析全流程 + 设备 REST 化

阶段	任务数	总工时	核心目标
P4 发布门禁	22	37h	认证去硬编码 + 安全 + 耗材 + GraphQL 退役
合计	68	134h	

按模块

模块	任务数	工时
患者管理	18	36.5h
透析治疗	15	33.5h
设备监控	7	16.5h
耗材管理	4	7.5h
医嘱管理	3	6.5h
认证	3	6h
排班管理	2	5h
仪表板	2	5h
多租户	2	4.5h
架构	2	2.5h
工程化	5	6h
验收	4	3h

B.5 依赖关系

Phase 1 — 患者后端闭环（纯后端，零切换）

- └─ LabReport: model → service → handler → restClient
- └─ Order: service → handler → restClient (model 已存在 treatment.go)
- └─ MonthlySummary: model → service → handler → restClient
- └─ /core 三处 TODO 真实化（依赖上面三组 API）

↓

Phase 2 — 患者前端 + 透析后端（依赖 P1）

- └─ 数据修复: Preload + 事务化
- └─ 4 个 Tab 接入 REST API
- └─ 透析子流程 5 组 CRUD (BeforeCheck/BeforeSigns/DuringParam/AfterSigns/Alarm)
- └─ PatientId 类型转换 + 治疗历史 API

↓

Phase 3 — 透析前端 + 设备全栈（依赖 P2）

- └─ HistoryTab → 9 Tab 完整闭环
- └─ DialysisProcessing.tsx 全部脱 Mock
- └─ 排班最小对接（透析依赖排班查询）
- └─ 设备: model → service → handler → restClient → 前端

↓

Phase 4 — 发布门禁（依赖 P3）

- └─ 认证 P0: bcrypt + JWT + 删除硬编码
- └─ 多租户: TenantId/CreatorId 去硬编码
- └─ 安全门禁: 角色权限 + 参数校验 + 审计日志
- └─ 耗材全栈 + 排班前端对接 + Dashboard 统计
- └─ GraphQL 完全退役 + 测试 + 迁移工具

B.6 风险管理

风险	影响	缓解策略
主键类型不一致 (UUID vs bigint)	跨域关联失败	P2 实现类型转换层
双 API 并存至 P4	维护成本翻倍	P4 分批退役 GraphQL
设备域从零搭建	模型设计可能不足	P3 最小可用优先
DialysisProcessing.tsx 1,487 行	对接量大	P3 预留分析时间
透析隐式依赖排班	排班未对接时不完整	P3 排班最小对接
测试服 vs 线上环境差异	HDIS 接口返工	先在线上验证再开发

未纳入本轮

功能	原因
用户管理 CRUD	无前端页面
处方管理	无产品需求
文书管理	零实现，需产品定义
LIS/第三方对接	外部依赖未就绪
AntD/TailwindCSS 统一	维持现状